Training neural networks: backpropagation, regularisation and initialisation

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

A multilayer perceptron (Note 11) is trained like every other model in this course: choose a loss, add a penalty, and run a stochastic gradient method (Note 7) on the empirical risk. What is new is the size and structure of the problem: millions of parameters arranged in layers, and an objective that is not convex. This note derives *backpropagation*, the algorithm that computes the gradient of a network at the cost of about two extra forward passes, first for one example, then for a mini-batch in matrix form, then for any computational graph (automatic differentiation). It then treats what makes networks train well or badly: checking gradients, weight decay and dropout, initialisation at the right scale, the choice of hyperparameters, plateaus and vanishing gradients, and the two modern remedies built into every large model, normalisation layers and residual connections. It ends with a debugging checklist.

1 Training is regularised risk minimisation

The network. We use the network of Note 11, Section 4.2, and its notation, recalled here: layers $k = 1, \dots, L$, input $\mathbf{h}^{(0)} = \mathbf{x} \in \mathbb{R}^d$, H_k units in layer k ($H_0 = d$), and

$$\mathbf{a}^{(k)} = W^{(k)}\mathbf{h}^{(k-1)} + \mathbf{b}^{(k)}, \qquad \mathbf{h}^{(k)} = g\big(\mathbf{a}^{(k)}\big) \quad (k < L), \qquad \mathbf{y} = o\big(\mathbf{a}^{(L)}\big), \tag{1}$$

with weight matrices $W^{(k)} \in \mathbb{R}^{H_k \times H_{k-1}}$, bias vectors $\mathbf{b}^{(k)} \in \mathbb{R}^{H_k}$, an element-wise activation g and an output function o (identity, sigmoid or softmax); $\mathbf{a}^{(k)}$ are the pre-activations and $\mathbf{h}^{(k)}$ the activations of layer k. All weights and biases together form the parameter vector θ, with p components (counted in Note 11, Section 4.2). Targets are written $\mathbf{t}$ (a one-hot vector for classification), outputs $\mathbf{y}$, as in Note 9.

The objective. Training minimises the regularised empirical risk of Note 4 (Sections 3.2 and 7.3), written for networks in Note 11, Section 7.1,

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell\big(\mathbf{y}(\mathbf{x}_i;\theta), \mathbf{t}_i\big) + \lambda\,\Omega(\theta), \tag{2}$$

with a per-example loss ℓ (a smooth surrogate of the error rate, Note 4, Section 2) and a penalty Ω. Note 7, Section 5, gave the algorithm: draw a mini-batch B, compute the gradient of the mini-batch objective, step against it. To apply it to a network we need four things, which organise this note:

1. the gradient of the loss with respect to the network's output (Section 2);
2. an efficient way to carry that gradient back to every weight and bias: backpropagation (Sections 3–5);
3. a penalty Ω and its gradient, and other regularisers specific to networks (Section 6);

4. a starting point θ_0 (Section 7).

Sections 8–10 discuss hyperparameters, why the optimisation is hard, and the practical and modern tools that make it work.

2 The gradient at the output

Backpropagation starts from the derivative of the loss of one example with respect to the output pre-activations,

$$\boldsymbol{\delta}^{(L)} = \frac{\partial \ell}{\partial \mathbf{a}^{(L)}} \in \mathbb{R}^{H_L}.$$

For an m-class classifier the output is the softmax $y_c = e^{a_c}/S$, with $S = \sum_{j=1}^{m} e^{a_j}$ and $\mathbf{a} = \mathbf{a}^{(L)}$, and the loss is the cross-entropy $\ell = -\sum_c t_c \log y_c = -\log y_{\text{true}}$. Note 9, Section 4.4 obtained the result quickly by writing $\ell = -a_{\text{true}} + \log S$. Here we take the longer road through the Jacobian of the softmax, because the same structure — a Jacobian times an upstream gradient — is what backpropagation does at every layer. Class indices are c and j; the letter k is kept for layers.

Derivation: the softmax Jacobian. By the quotient rule, with $\partial e^{a_c}/\partial a_j = \mathbf{1}\{c = j\}\, e^{a_c}$ and $\partial S/\partial a_j = e^{a_j}$,

$$\frac{\partial y_c}{\partial a_j} = \frac{\mathbf{1}\{c = j\}\, e^{a_c} S - e^{a_c} e^{a_j}}{S^2} = \frac{e^{a_c}}{S}\Big(\mathbf{1}\{c = j\} - \frac{e^{a_j}}{S}\Big) = y_c\,\big(\mathbf{1}\{c = j\} - y_j\big).$$

In matrix form the Jacobian is $\partial \mathbf{y}/\partial \mathbf{a} = \operatorname{diag}(\mathbf{y}) - \mathbf{y}\mathbf{y}^{\mathsf{T}}$: symmetric, and every row sums to $y_c - y_c \sum_j y_j = 0$ (adding the same constant to all scores does not change the softmax).

Derivation: the chain rule through the softmax. The loss depends on a_j through all the y_c:

$$\frac{\partial \ell}{\partial a_j} = \sum_{c=1}^{m} \frac{\partial \ell}{\partial y_c}\,\frac{\partial y_c}{\partial a_j} = \sum_c \Big(-\frac{t_c}{y_c}\Big)\, y_c\,\big(\mathbf{1}\{c = j\} - y_j\big) = -\sum_c t_c\, \mathbf{1}\{c = j\} + y_j \sum_c t_c = -t_j + y_j,$$

using $\sum_c t_c = 1$. The $1/y_c$ of the logarithm cancels the factor y_c of the Jacobian, and

$$\boldsymbol{\delta}^{(L)} = \mathbf{y} - \mathbf{t}. \tag{3}$$

> **Worked example — 1. The softmax gradient with exact fractions**
>
> Take $m = 3$ scores $\mathbf{a} = (0, \ln 3, \ln 4)$, so that $e^{\mathbf{a}} = (1, 3, 4)$, $S = 8$ and $\mathbf{y} = (\frac{1}{8}, \frac{3}{8}, \frac{1}{2})$. Let the true class be the second, $\mathbf{t} = (0, 1, 0)$. The loss is $-\ln \frac{3}{8} = \ln \frac{8}{3} = 0.981$.
> *Via the Jacobian.* $\operatorname{diag}(\mathbf{y}) - \mathbf{y}\mathbf{y}^{\mathsf{T}}$ has rows $(\frac{7}{64}, -\frac{3}{64}, -\frac{1}{16})$, $(-\frac{3}{64}, \frac{15}{64}, -\frac{3}{16})$ and $(-\frac{1}{16}, -\frac{3}{16}, \frac{1}{4})$; each row sums to zero. The upstream gradient is $\partial \ell/\partial \mathbf{y} = (0, -\frac{1}{y_2}, 0) = (0, -\frac{8}{3}, 0)$, so $\boldsymbol{\delta}^{(L)}$ is $-\frac{8}{3}$ times the second row (equal to the second column, the Jacobian being symmetric): $-\frac{8}{3}(-\frac{3}{64}, \frac{15}{64}, -\frac{3}{16}) = (\frac{1}{8}, -\frac{5}{8}, \frac{1}{2})$.
> *Via* (3). $\mathbf{y} - \mathbf{t} = (\frac{1}{8}, \frac{3}{8} - 1, \frac{1}{2}) = (\frac{1}{8}, -\frac{5}{8}, \frac{1}{2})$, the same. A gradient step lowers a_1 and a_3 and raises the true score a_2; the components sum to zero.

Other output units. For the other matched pairs of Note 11, Section 5 (identity output with squared loss, sigmoid output with binary cross-entropy) the same cancellation gives $\boldsymbol{\delta}^{(L)} = \mathbf{y} - \mathbf{t}$, and that section also showed why a mismatched pair such as a sigmoid with squared loss trains badly. The backward pass therefore starts from (3) for every task. In code the loss and (3) are computed together from the logits $\mathbf{a}$, never from the probabilities (Note 11, Section 5).

3 Backpropagation

3.1 The chain rule on a computational graph

Any computation can be drawn as a directed acyclic graph: inputs and parameters are source nodes, each internal node applies one simple operation to its parents, and one node at the end holds the loss. If a variable u influences the loss only through the nodes $v_1, \dots, v_m$ that use it (its *children*), the multivariate chain rule reads

$$\frac{\partial \ell}{\partial u} = \sum_{j=1}^{m} \frac{\partial \ell}{\partial v_j} \frac{\partial v_j}{\partial u}. \tag{4}$$

Write $\bar{u} = \partial \ell / \partial u$ for this *adjoint*. Equation (4) says that the adjoint of a node is a sum over its children, each child contributing its own adjoint times a *local* derivative that involves only the child's operation. So the adjoints can be computed in one sweep from the loss back to the inputs, if the nodes are visited in reverse order and the local derivatives are evaluated at the values stored during the forward sweep. That is backpropagation [21], or reverse-mode automatic differentiation; the method was known in numerical analysis before it became the workhorse of neural networks [2].

> **Worked example — 2. Reverse mode on a small graph**
>
> Let $f(x, y) = (xy + x)^2$, computed as $z = xy$, $s = z + x$, $f = s^2$ (Figure 1). The input x is used twice, so it has two children.
> *Forward sweep* at $x = 1$, $y = 2$: $z = 2$, $s = 3$, $f = 9$.
> *Backward sweep.* Start with $\bar{f} = 1$. Then $\bar{s} = \bar{f} \cdot 2s = 6$; $\bar{z} = \bar{s} \cdot 1 = 6$; $\bar{y} = \bar{z} \cdot x = 6$; and x collects from both children, $\bar{x} = \bar{s} \cdot 1 + \bar{z} \cdot y = 6 + 12 = 18$.
> *Check.* $f = x^2(y + 1)^2$, so $\partial f / \partial x = 2x(y + 1)^2 = 18$ and $\partial f / \partial y = 2x^2(y + 1) = 6$. One backward sweep, costing about as much as the forward sweep, produced *both* partial derivatives. Forward-mode differentiation, which pushes derivatives from the inputs forwards, would need one sweep per input; finite differences two evaluations per input (Note 7, Section 6.5).

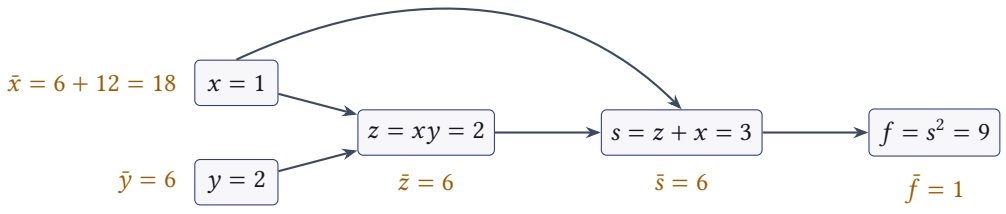


Figure 1. The computational graph of Worked example 2. Values (black) are computed left to right, adjoints (amber) right to left. The input x feeds two nodes, so its adjoint is the sum of two contributions.

3.2 The layer recursion

Apply (4) to the layered network (1), for one example. Define the error signal of layer k as the adjoint of its pre-activations, $\boldsymbol{\delta}^{(k)} = \partial \ell / \partial \mathbf{a}^{(k)}$; at the top it is (3).

Derivation: from one layer to the one below. The activation $h_j^{(k-1)}$ feeds every pre-activation of layer k, through $a_i^{(k)} = \sum_j W_{ij}^{(k)} h_j^{(k-1)} + b_i^{(k)}$, so $\partial a_i^{(k)}/\partial h_j^{(k-1)} = W_{ij}^{(k)}$ and by (4)

$$\frac{\partial \ell}{\partial h_j^{(k-1)}} = \sum_{i=1}^{H_k} \delta_i^{(k)} W_{ij}^{(k)}, \qquad \text{i.e.} \qquad \frac{\partial \ell}{\partial \mathbf{h}^{(k-1)}} = W^{(k)\mathsf{T}} \boldsymbol{\delta}^{(k)}.$$

Next, $h_j^{(k-1)} = g(a_j^{(k-1)})$ depends on $a_j^{(k-1)}$ alone, so the chain rule has a single term: $\delta_j^{(k-1)} = g'(a_j^{(k-1)})\, \partial \ell/\partial h_j^{(k-1)}$. Together, with $\odot$ the element-wise product,

$$\boldsymbol{\delta}^{(k-1)} = g'\big(\mathbf{a}^{(k-1)}\big) \odot \big(W^{(k)\mathsf{T}} \boldsymbol{\delta}^{(k)}\big), \qquad k = L, L-1, \dots, 2. \tag{5}$$

The error travels down through the *transposed* weights — the forward pass uses $W^{(k)}$, the backward pass $W^{(k)\mathsf{T}}$ — and is gated at every unit by the slope of its activation.

Derivation: the parameter gradients. The weight $W_{ij}^{(k)}$ appears only in $a_i^{(k)}$, with $\partial a_i^{(k)}/\partial W_{ij}^{(k)} = h_j^{(k-1)}$, and $\partial a_i^{(k)}/\partial b_i^{(k)} = 1$. Hence

$$\frac{\partial \ell}{\partial W_{ij}^{(k)}} = \delta_i^{(k)}\, h_j^{(k-1)}, \qquad \nabla_{W^{(k)}} \ell = \boldsymbol{\delta}^{(k)} \mathbf{h}^{(k-1)\mathsf{T}}, \qquad \nabla_{\mathbf{b}^{(k)}} \ell = \boldsymbol{\delta}^{(k)}. \tag{6}$$

The gradient of a weight is (error at its output end) × (activation at its input end): an outer product, of the same shape as $W^{(k)}$. For a network with no hidden layer this is the "prediction minus target, times input" of logistic regression (Note 9, Section 4.2).

Activation derivatives. The recursion needs g' at the stored pre-activations. Note 11, Section 2, tabulated and plotted these derivatives, and noted that $\sigma' = \sigma(1-\sigma)$ and $\tanh' = 1 - \tanh^2$ follow from the stored outputs $h = g(a)$ alone. What matters here is their *size*: $\sigma' \le \frac{1}{4}$ (largest at $a = 0$), $\tanh' \le 1$, ReLU$' \in \{0, 1\}$, and the derivatives of GELU and SiLU lie in $[-0.129, 1.129]$ and about $[-0.100, 1.100]$. ReLU is not differentiable at $a = 0$; libraries use $g'(0) = 0$, a valid subgradient, and the choice almost never matters because exact zeros are rare. Section 9.2 shows that these sizes decide whether a deep network can be trained at all.

3.3 A complete forward and backward pass

Worked example — 3. One training step of a 2–2–1 network

A regression network with two ReLU hidden units and a linear output, squared loss $\ell = \frac{1}{2}(y-t)^2$:

$$W^{(1)} = \begin{pmatrix} 0.5 & -0.5 \\ 1 & 0.5 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \quad W^{(2)} = (1 \;\; 2), \quad b^{(2)} = 0, \qquad \mathbf{x} = (1, 2), \;\; t = 1.$$

Forward. $\mathbf{a}^{(1)} = (0.5{-}1{+}0,\; 1{+}1{-}1) = (-0.5, 1)$, so $\mathbf{h}^{(1)} = (0, 1)$: unit 1 is off. $y = 1{\cdot}0{+}2{\cdot}1{+}0 = 2$ and $\ell = \frac{1}{2}(2-1)^2 = 0.5$. Store $\mathbf{x}$, $\mathbf{a}^{(1)}$, $\mathbf{h}^{(1)}$.
Backward, output layer. $\delta^{(2)} = y - t = 1$. By (6), $\nabla_{W^{(2)}} \ell = \delta^{(2)} \mathbf{h}^{(1)\mathsf{T}} = (0, 1)$ and $\partial \ell/\partial b^{(2)} = 1$.
Backward, hidden layer. $W^{(2)\mathsf{T}} \delta^{(2)} = (1, 2)$; the ReLU gate $g'(\mathbf{a}^{(1)}) = (0, 1)$ gives $\boldsymbol{\delta}^{(1)} = (0, 2)$. Then

$$\nabla_{W^{(1)}} \ell = \boldsymbol{\delta}^{(1)} \mathbf{x}^{\mathsf{T}} = \begin{pmatrix} 0 \\ 2 \end{pmatrix} (1 \;\; 2) = \begin{pmatrix} 0 & 0 \\ 2 & 4 \end{pmatrix}, \qquad \nabla_{\mathbf{b}^{(1)}} \ell = (0, 2).$$

The inactive unit receives no gradient at all, neither on its incoming weights nor on its bias. *Update* with $\eta = 0.05$: $W^{(1)}$ becomes $\begin{pmatrix} 0.5 & -0.5 \\ 0.9 & 0.3 \end{pmatrix}$, $\mathbf{b}^{(1)} = (0, -1.1)$, $W^{(2)} = (1, 1.95)$, $b^{(2)} = -0.05$. A new forward pass gives $\mathbf{a}^{(1)} = (-0.5, 0.4)$, $y = 1.95 \times 0.4 - 0.05 = 0.73$ and $\ell = \frac{1}{2}(0.27)^2 = 0.036$: the loss fell from 0.5 to 0.036. (With $\eta = 0.1$ the step overshoots and switches unit 2 off as well; Question B5.) A central difference with $h = 0.01$ on $W^{(1)}_{22}$ gives 4.000, confirming the largest component of the gradient (Section 4).

3.4 Mini-batches in matrix form

In practice a mini-batch of n' examples is processed at once. Stack the examples as rows: $H^{(0)} = X_B \in \mathbb{R}^{n' \times d}$, and let $A^{(k)}, H^{(k)} \in \mathbb{R}^{n' \times H_k}$ hold the pre-activations and activations, one row per example. Row i of $A^{(k)}$ is the transpose of $W^{(k)}\mathbf{h}_i^{(k-1)} + \mathbf{b}^{(k)}$, that is $\mathbf{h}_i^{(k-1)\mathsf{T}} W^{(k)\mathsf{T}} + \mathbf{b}^{(k)\mathsf{T}}$; stacking the rows, the forward pass (1) becomes $A^{(k)} = H^{(k-1)} W^{(k)\mathsf{T}} + \mathbf{1}\mathbf{b}^{(k)\mathsf{T}}$, a matrix product plus the bias added to every row. For the mean loss over the batch each row of the output error is divided by n', $\Delta^{(L)} = (Y - T)/n'$, where row i of $\Delta^{(k)}$ is $\boldsymbol{\delta}_i^{(k)\mathsf{T}}$. Transposing $W^{(k)\mathsf{T}}\boldsymbol{\delta}_i^{(k)}$ row by row gives $\boldsymbol{\delta}_i^{(k)\mathsf{T}} W^{(k)}$, so (5)–(6) become

$$\nabla_{W^{(k)}} = \Delta^{(k)\mathsf{T}} H^{(k-1)}, \qquad \nabla_{\mathbf{b}^{(k)}} = \Delta^{(k)\mathsf{T}} \mathbf{1}, \qquad \Delta^{(k-1)} = \big(\Delta^{(k)} W^{(k)}\big) \odot g'\big(A^{(k-1)}\big). \tag{7}$$

The product $\Delta^{(k)\mathsf{T}} H^{(k-1)} = \sum_{i \in B} \boldsymbol{\delta}_i^{(k)} \mathbf{h}_i^{(k-1)\mathsf{T}}$ sums the per-example outer products of (6), so the batch gradient is the average of the per-example gradients, as it must be. Algorithm 1 collects the steps.

Algorithm 1 Forward and backward pass of an L-layer network on a mini-batch

Require: mini-batch $X_B \in \mathbb{R}^{n' \times d}$, targets $T \in \mathbb{R}^{n' \times H_L}$; parameters $W^{(k)}, \mathbf{b}^{(k)}$, $k = 1, \dots, L$; activation g; output/loss pair with $\Delta^{(L)} = (Y - T)/n'$

function ForwardBackward(X_B, T)
  $H^{(0)} \leftarrow X_B$
  for $k = 1, \dots, L$ **do** ▷ forward pass; keep every $A^{(k)}$, $H^{(k)}$
    $A^{(k)} \leftarrow H^{(k-1)} W^{(k)\mathsf{T}} + \mathbf{1}\mathbf{b}^{(k)\mathsf{T}}$ ▷ $O(n' H_k H_{k-1})$
    $H^{(k)} \leftarrow g(A^{(k)})$ if $k < L$, else $Y \leftarrow o(A^{(L)})$
  end for
  $J_B \leftarrow \frac{1}{n'} \sum_{i \in B} \ell(\mathbf{y}_i, \mathbf{t}_i)$; $\quad \Delta \leftarrow (Y - T)/n'$ ▷ (3)
  for $k = L, L-1, \dots, 1$ **do** ▷ backward pass
    $G_W^{(k)} \leftarrow \Delta^{\mathsf{T}} H^{(k-1)}$; $\quad G_b^{(k)} \leftarrow \Delta^{\mathsf{T}} \mathbf{1}$ ▷ $O(n' H_k H_{k-1})$
    if $k > 1$ **then**
      $\Delta \leftarrow (\Delta W^{(k)}) \odot g'(A^{(k-1)})$ ▷ $O(n' H_k H_{k-1})$
    end if
  end for
  return J_B and the gradients $\{G_W^{(k)}, G_b^{(k)}\}_{k=1}^{L}$
end function

Running time — forward and backward pass

Time. The forward pass multiplies an $n' \times H_{k-1}$ matrix by an $H_{k-1} \times H_k$ matrix in each layer, $n' H_k H_{k-1}$ multiply-adds; summed over layers that is about $n' p$. The backward pass does *two* products of the same size per layer ($\Delta^{\mathsf{T}} H$ for the weights, ΔW for the layer below; the first layer skips the second), so a training step costs about three forward passes, $\approx 3n' p$ multiply-adds; the element-wise g, g' and $\odot$ add only $O(n' \sum_k H_k)$. *Prediction*: one forward pass, $O(p)$ per example. *Memory*: the parameters and their gradients, $2p$ numbers, plus the stored activations, $O(n' \sum_k H_k)$, which grow with the batch size and the

depth. Example: the network 784–256–256–10 has $p = 269\,322$ parameters, so a batch of $n' = 128$ costs about $3 \times 128 \times 269\,322 \approx 1.0 \times 10^8$ multiply-adds per step.

3.5 Automatic differentiation

Nothing in Section 3.1 required layers. Modern software represents the computation as a graph of *modules* (matrix product, addition, ReLU, softmax-cross-entropy, convolution, attention, ...), each providing two methods: FPROP computes the module's output from its inputs, and BPROP receives the adjoint of the output and returns the adjoints of the inputs, using values saved during FPROP. Calling FPROP in topological order and BPROP in the reverse order is backpropagation for any architecture (Algorithm 2). The layer recursion (5) is the special case of a chain of "affine" and "activation" modules.

Algorithm 2 Reverse-mode automatic differentiation on a computational graph

Require: nodes $v_1, \dots, v_N$ in topological order (each node's parents come before it), $v_N = \ell$ a scalar; each node has FPROP and BPROP

function BACKPROP(inputs, parameters)
  for $j = 1, \dots, N$ **do**
    $v_j \leftarrow v_j$.FPROP(values of the parents of v_j) ▷ save what BPROP needs
  end for
  $\bar{v}_j \leftarrow 0$ for all j; $\bar{v}_N \leftarrow 1$
  for $j = N, N-1, \dots, 1$ **do**
    for each parent u of v_j **do**
      $\bar{u} \leftarrow \bar{u} + v_j$.BPROP$(\bar{v}_j, u)$ ▷ $\bar{v}_j\, \partial v_j / \partial u$; accumulate, (4)
    end for
  end for
  return $\bar{u}$ for every parameter u
end function

Running time — reverse-mode automatic differentiation

Time: each BPROP costs a small constant times its FPROP (one or two operations of the same size), so the full gradient costs a small constant (typically 2 to 4) times one evaluation of ℓ, *whatever the number of parameters*; forward mode or finite differences would multiply the cost by the number of parameters. *Memory*: every value that a BPROP needs is kept from the forward sweep until the backward sweep reaches it, so memory grows with the size of the graph, that is with batch size times depth. The accumulation $\bar{u} \leftarrow \bar{u} + \dots$ is what makes shared parameters (one weight used in several places) correct.

Modern connection — autograd, and trading time for memory

PyTorch, JAX and TensorFlow implement Algorithm 2: the user writes only the forward computation and calls `loss.backward()` (PyTorch) or `jax.grad` (JAX); the graph is recorded while the forward code runs, so loops and branches in ordinary Python are differentiated too. For large models the bottleneck is the memory of the stored activations rather than the arithmetic. *Activation checkpointing* keeps only some intermediate values and recomputes the others during the backward sweep; keeping one checkpoint every $\sqrt{L}$ layers of an L-layer chain reduces the activation memory from $O(L)$ to $O(\sqrt{L})$ for the price of roughly one extra forward pass [6].

In physics — backpropagation is the adjoint method

Physicists met reverse-mode differentiation long before deep learning, as the *adjoint-state method*. In seismic imaging, weather forecasting or optimal control, a misfit between simulation and observations depends on millions of model parameters (for instance the wave speed in every cell of a grid). One forward simulation and one simulation of the adjoint equations, run backwards in time with the residuals as sources, give the gradient with respect to all of them [17]; Lagrange multipliers of the dynamical constraints play the role of the adjoints $\bar{u}$. Backpropagation is the same construction for a discrete-time "dynamics" $\mathbf{h}^{(k-1)} \to \mathbf{h}^{(k)}$, and differentiable simulators now combine the two.

4 Checking gradients

A wrong gradient rarely causes a crash: training simply proceeds more slowly or to a worse solution. Before trusting a hand-written backward pass (or a custom module), compare it with central differences, which Note 7, Section 6.5, introduced together with their $O(h^2)$ error. Two practical points matter for networks. First, compare with a *relative* error,

$$r = \frac{\left\| \mathbf{g}_{\text{bp}} - \mathbf{g}_{\text{fd}} \right\|}{\left\| \mathbf{g}_{\text{bp}} \right\| + \left\| \mathbf{g}_{\text{fd}} \right\|}, \tag{8}$$

because gradients of different layers can differ by orders of magnitude. In double precision, $r \lesssim 10^{-7}$ means correct, $r \gtrsim 10^{-3}$ almost certainly means a bug. Second, a network has too many parameters to check them all; a few random coordinates per parameter tensor suffice, on a tiny batch (Algorithm 3).

Algorithm 3 Gradient checking

Require: loss $J(\theta)$ on a fixed tiny batch (dropout off, double precision); backprop gradient $\mathbf{g}_{\text{bp}}$; step h; number m of coordinates to test per parameter tensor

1: **function** GradCheck(θ)
2: **for** each parameter tensor; for m randomly chosen coordinates k in it **do**
3: $g_{\text{fd},k} \leftarrow \big(J(\theta + h\mathbf{e}_k) - J(\theta - h\mathbf{e}_k)\big)/(2h)$ ▷ two forward passes
4: **end for**
5: **return** r of (8), restricted to the tested coordinates
6: **end function**

Running time — gradient checking

Two forward passes per tested coordinate: $O(m\, L_{\text{t}}\, n'\, p)$ for L_{t} parameter tensors, against $O(n'\, p)$ for the whole backprop gradient. Checking all p coordinates would cost $O(n'\, p^2)$, which is why it is done only on small networks or a sample of coordinates, once, while debugging. Memory: $O(p)$.

Worked example — 4. Catching a forgotten derivative

Take a 2–2–1 network with tanh hidden units and a sigmoid output with cross-entropy, at $W^{(1)} = \begin{pmatrix} 0.5 & 0.2 \\ -0.3 & 0.8 \end{pmatrix}$, $\mathbf{b}^{(1)} = (0.1, 0)$, $W^{(2)} = (1, -1)$, $b^{(2)} = 0$, for the example $\mathbf{x} = (1, -1)$, $t = 1$. The forward pass gives $\mathbf{a}^{(1)} = (0.4, -1.1)$, $\mathbf{h}^{(1)} = (0.380, -0.800)$, output $y = 0.765$ and loss 0.268. Central differences on all nine parameters agree with backprop to $r = 9.1 \times 10^{-6}$ for $h = 10^{-2}$ and $r = 1.1 \times 10^{-10}$ for $h = 10^{-6}$ (the $O(h^2)$ behaviour and the rounding floor of Note 7, Section 6.5).

Now introduce a typical bug: forget the factor $g'(\mathbf{a}^{(1)}) = 1 - \tanh^2 = (0.856, 0.359)$ in (5). Compare two coordinates:

	correct (and finite difference)	buggy	ratio
$\partial \ell / \partial W_{11}^{(1)}$	-0.2011	-0.2350	$1/0.856$
$\partial \ell / \partial W_{21}^{(1)}$	0.0844	0.2350	$1/0.359$

Over all nine parameters $r = 0.23$: unmistakable, although the buggy gradient still points roughly downhill and training would "work". The three output-layer coordinates agree exactly, which localises the bug to the step that carries the error into the hidden layer.

Pitfalls: with ReLU, a pre-activation within h of zero makes the finite difference straddle a kink, so a few coordinates may disagree although the code is right; dropout and data augmentation must be off (otherwise J is random); and single precision cannot resolve r below about 10^{-3} to 10^{-4}.

5 The training loop

With gradients in hand, training is the mini-batch loop of Note 7, Section 5.3, with the regularisers of the next section and the early stopping with patience of Note 7, Section 7.1. Algorithm 4 writes it out for plain SGD with weight decay; replacing its parameter update by the AdamW step of Note 7, Section 6.4, gives the default for large models.

Algorithm 4 Training a network with mini-batch SGD, weight decay and early stopping

Require: training and validation sets; initial θ (Algorithm 6); learning rates η_t; weight decay λ; batch size n'; patience P; maximum number of epochs E

function TrainNetwork(θ)
  $\theta_{\text{best}} \leftarrow \theta$, $\ell_{\text{best}} \leftarrow \infty$, $c \leftarrow 0$, $t \leftarrow 0$
  for epoch $= 1, \dots, E$ **do**
    for each mini-batch B of a fresh random permutation of the training set **do**
      $(J_B, \mathbf{g}) \leftarrow$ ForwardBackward(X_B, T_B) ▷ Algorithm 1, $\approx 3n'p$
      add $\lambda W^{(k)}$ to the gradient of every weight matrix (not to the biases) ▷ Section 6.1
      $\theta \leftarrow \theta - \eta_t \mathbf{g}$; $t \leftarrow t + 1$ ▷ or momentum / AdamW (Note 7)
    end for
    $\ell_{\text{v}} \leftarrow$ validation loss; record training and validation curves ▷ forward passes only
    if $\ell_{\text{v}} < \ell_{\text{best}}$ **then** $\ell_{\text{best}} \leftarrow \ell_{\text{v}}$, $\theta_{\text{best}} \leftarrow \theta$, $c \leftarrow 0$
    else $c \leftarrow c + 1$; **if** $c = P$ **then break**
    end if
  end for
  return θ_{best}
end function

Running time — training a network

Per epoch: n/n' steps at $\approx 3n'p$ each, i.e. $\approx 3np$ multiply-adds, plus $n_{\text{valid}}\, p$ for the validation pass; E epochs cost $O(Enp)$. *Memory*: θ, its gradient and θ_{best} ($3p$ numbers; $+2p$ for AdamW's moments) plus $O(n' \sum_k H_k)$ activations. *Prediction*: $O(p)$ per example, independent of n — unlike kernel methods (Note 10), the training set is not needed after training.

Modern connection — mixed-precision training

Most large networks are trained in *mixed precision* [15]: matrix products run in 16-bit floating point, which modern accelerators execute several times faster than 32-bit and which halves the activation memory, while a 32-bit master copy of the weights receives the updates (an update ηg is often too small to change a 16-bit weight). The IEEE half-precision format (fp16) has only 5 exponent bits, so its largest value is 65 504 and gradients below about 6×10^{-8} underflow to zero; the loss is therefore multiplied by a large *loss scale* before the backward pass and the gradients divided by it afterwards — the chain rule is linear in $\bar{\ell}$, so this changes nothing but the floating-point range. The bfloat16 format keeps the 8 exponent bits of fp32 (and only 7 mantissa bits), so it rarely needs loss scaling.

6 Regularisation for networks

Networks are flexible enough to fit their training set perfectly, so capacity control (Note 4, Sections 5–7) is essential. Note 11, Section 7.2, showed an unregularised network on the two-moons data memorising its training set while its validation loss climbs. The centre panel of Figure 2 repeats the experiment on only 40 points and adds the remedy of the next subsection.

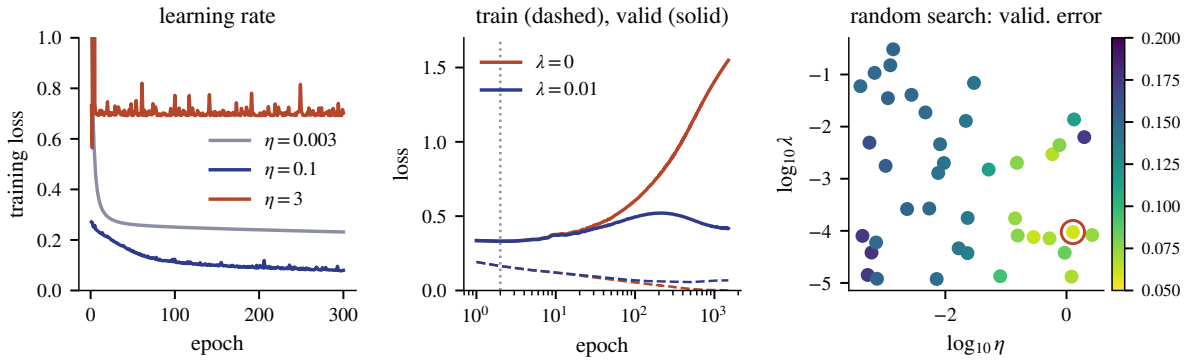


Figure 2. Training ReLU networks on the two-moons data with Algorithm 4 without early stopping (fixed numbers of epochs; He initialisation, batches of 20 or 10). Left: training loss of a 2–64–64–1 network on 200 points for three learning rates; $\eta = 0.003$ is slow (0.232 after 300 epochs), $\eta = 0.1$ reaches 0.080, and $\eta = 3$ jumps to a loss of 10.1 in the third epoch and then stays near 0.70: every unit of the second hidden layer is dead, and the network outputs the same probability, 0.40, for every input (training error 47.5%). Centre: a 2–128–128–1 network on only 40 points ($\eta = 0.05$, log scale in epochs). Without weight decay the training loss falls to 0.001 and the validation loss rises to 1.55 (validation error 16.8%); with $\lambda = 0.01$ it ends at 0.42 (error 13.6%). The validation loss is lowest, 0.33, already at epoch 2 (dotted line), while the validation error of the $\lambda = 0$ run keeps falling to its minimum, 14.9%, at epoch 202. Right: 40 trials of random search over $\log_{10}\eta$ and $\log_{10}\lambda$ (60 epochs each, 2000 validation points); colour is validation error, the best trial (circled, 6.05%) has $\eta \approx 1.3$, $\lambda \approx 10^{-4}$.

6.1 Weight decay

The L_2 penalty $\Omega(\theta) = \frac{1}{2}\sum_k \left\|W^{(k)}\right\|_F^2 = \frac{1}{2}\sum_k \sum_{i,j} \left(W_{ij}^{(k)}\right)^2$ (the factor $\frac{1}{2}$ only simplifies the gradient; Note 11, Section 7.1, omits it, so its λ is half of ours) adds $\lambda W^{(k)}$ to each weight gradient, and a gradient step becomes

$$W^{(k)} \leftarrow W^{(k)} - \eta\left(\nabla_{W^{(k)}}\ell + \lambda W^{(k)}\right) = (1-\eta\lambda)\, W^{(k)} - \eta\, \nabla_{W^{(k)}}\ell \ : \tag{9}$$

before each step every weight is multiplied by $1-\eta\lambda$, hence the name *weight decay*. It is ridge regression's penalty (Note 8, Section 4.3) applied to every layer, and the MAP estimate under a Gaussian prior on the weights (Note 8, Section 6). Two network-specific points:

- *Biases are not penalised.* A bias shifts a unit's threshold; it does not make the function steeper or more complex, and penalising it would pull outputs towards zero for no reason (for instance away from the mean of the targets). Normalisation layers' scale and shift parameters (Section 10.2) are usually exempt for the same reason.
- *With adaptive optimisers the decay must be decoupled.* Inside Adam the term λW would be rescaled per parameter; AdamW applies (9)'s shrinkage directly (Note 7, Section 6.4) [14].

In the centre panel of Figure 2, $\lambda = 0.01$ keeps the validation loss at 0.42 instead of 1.55, and the final validation error at 13.6% instead of 16.8%.

6.2 The L_1 penalty

$\Omega(\theta) = \sum_k \sum_{i,j} |W_{ij}^{(k)}|$ has the (sub)gradient $\operatorname{sign}(W^{(k)})$, so each step moves every weight towards zero by the *constant* amount $\eta\lambda$, however small the weight already is. Weights whose data gradient is weak reach zero. With the soft-thresholding update of Note 8, Section 5.1 (subtract $\eta\lambda$ in absolute value and clip at zero) they stay exactly at zero, as in the lasso; a plain subgradient step instead makes them oscillate around zero. The corresponding prior is the Laplace distribution. L_1 penalties on weights are used for pruning and interpretability, more rarely for accuracy.

> **Worked example — 5. Shrinking versus clipping to zero**
>
> Take one weight $w = 0.5$ whose data gradient is zero (it does not help the fit), $\eta = 0.1$ and $\lambda = 0.5$. L_2: each step multiplies w by $1 - \eta\lambda = 0.95$; after ten steps $w = 0.5 \times 0.95^{10} = 0.299$, and it never reaches zero. L_1 with soft thresholding: each step subtracts $\eta\lambda = 0.05$; after ten steps $w = 0$, and the clip keeps it there instead of letting an eleventh step overshoot to -0.05.

6.3 Dropout

Dropout [24] regularises by injecting noise. During training, each hidden unit's output is set to zero with probability p_d, independently for each unit, each example and each step, and the surviving units are scaled by $1/(1-p_\mathrm{d})$ ("inverted" dropout). With the mask $m_j \sim \mathrm{Bernoulli}(1-p_\mathrm{d})$,

$$\tilde{h}_j = \frac{m_j\, h_j}{1-p_\mathrm{d}}, \qquad \mathbb{E}[\tilde{h}_j] = \frac{(1-p_\mathrm{d})\, h_j}{1-p_\mathrm{d}} = h_j,$$

so the expected input of the next layer is unchanged, and at test time the network is used as it is, without masks or rescaling. A unit can no longer rely on the presence of particular partners, which discourages fragile co-adaptations; one can also view training as fitting an exponentially large family of thinned networks that share weights, and the test-time network as an approximation to their average. Backpropagation through dropout multiplies the upstream gradient by the same mask (Algorithm 5): a dropped unit passes no gradient.

Algorithm 5 Inverted dropout on a layer's activations

Require: activations $H \in \mathbb{R}^{n' \times H_k}$; drop probability $p_{\mathrm{d}} \in [0, 1)$; flag *training*

function DropoutForward(H)
  if not *training* **then return** H
  end if ▷ inference: identity
  $M_{ij} \leftarrow \mathbf{1}\{u_{ij} > p_{\mathrm{d}}\}/(1 - p_{\mathrm{d}})$ with $u_{ij} \sim \mathcal{U}(0, 1)$ ▷ store M
  return $M \odot H$
end function
function DropoutBackward($\bar{H}_{\mathrm{out}}$) ▷ adjoint of the output
  return $M \odot \bar{H}_{\mathrm{out}}$
end function

Running time — dropout

$O(n' H_k)$ random numbers and multiplications per layer and step, negligible next to the $O(n' H_k H_{k-1})$ matrix product; memory $O(n' H_k)$ for the mask (often regenerated from the random seed instead of being stored). Zero cost at prediction. Training usually needs more epochs, because the gradients are noisier.

Worked example — 6. A dropout sample

Let $\mathbf{h} = (2, 0, 1, 3)$ and $p_{\mathrm{d}} = 0.5$. The mask $(1, 0, 0, 1)$ gives $\tilde{\mathbf{h}} = (4, 0, 0, 6)$. For a next-layer unit with weights $(1, -1, 2, 1)$ the pre-activation is $4 + 0 + 0 + 6 = 10$ for this sample, while its expectation is the undropped value $2 - 0 + 2 + 3 = 7$. Each $m_j/(1 - p_{\mathrm{d}})$ takes the values 0 and 2 with equal probability, mean 1 and variance 1, so the dropout noise has variance $\sum_j w_j^2 h_j^2 = 4 + 0 + 4 + 9 = 17$ (standard deviation 4.1). The network must learn to classify correctly despite this noise.

6.4 Early stopping and the bias–variance view

All these penalties trade a little bias for a lower variance in the sense of Note 4, Section 6. The number of training epochs is a capacity control of its own (Note 11, Section 7.2), and early stopping with patience (Note 7, Section 7.1) is already part of Algorithm 4: it returns the parameters θ_{best} of the epoch with the lowest validation loss, not the last ones. What is monitored matters. In the centre panel of Figure 2 the validation loss is lowest at epoch 2, because the cross-entropy punishes the network's growing confidence on the points it gets wrong, while the validation error improves until about epoch 200; monitoring the error, or using a patience of many epochs, avoids stopping too early. In practice one combines weight decay with early stopping, and adds dropout when the network is large relative to the data. Data augmentation, the most effective regulariser for images, is treated in Note 13, Section 6.

7 Initialisation

Biases. Biases are initialised to zero (or a small positive constant for ReLU units, so that they start active). The interesting question is the weights.

Derivation: zero weights never move. Suppose all weights are zero in a network with tanh hidden units and zero biases. Then every $\mathbf{a}^{(k)} = \mathbf{0}$, every hidden activation $\tanh 0 = 0$, and the output does not depend on the input. In the backward pass, (6) gives $\nabla_{W^{(k)}} \ell = \boldsymbol{\delta}^{(k)} \mathbf{h}^{(k-1)\mathsf{T}} = 0$ for every $k \geq 2$,

because $\mathbf{h}^{(k-1)} = \mathbf{0}$; and (5) gives $\boldsymbol{\delta}^{(k-1)} = g'(\mathbf{0}) \odot (W^{(k)\mathsf{T}} \boldsymbol{\delta}^{(k)}) = \mathbf{0}$, because $W^{(k)} = 0$, so $\nabla_{W^{(1)}} \ell = 0$ too. Only the output bias gets a gradient. Gradient descent therefore never leaves the set where all weights and hidden biases are zero: only the output bias moves, to the constant that best fits the targets, and the point it reaches is a stationary point (a saddle) of J. With sigmoid hidden units $h = \sigma(0) = \frac{1}{2} \neq 0$, so the top-layer weights do receive gradients; but all units of a layer stay identical, which is the symmetry problem treated next.

Derivation: equal weights stay equal. Suppose instead that two hidden units i and i' of layer k start with identical incoming weights and biases and identical outgoing weights. Then $a_i^{(k)} = a_{i'}^{(k)}$ for every input, hence $h_i^{(k)} = h_{i'}^{(k)}$; in the backward pass $\delta_i^{(k)} = g'(a_i^{(k)}) \sum_r W_{ri}^{(k+1)} \delta_r^{(k+1)} = \delta_{i'}^{(k)}$, so by (6) their incoming weights receive identical gradients, and so do their outgoing weights ($\delta_r^{(k+1)} h_i^{(k)} = \delta_r^{(k+1)} h_{i'}^{(k)}$). After the update they are still identical, and by induction forever: the two units compute the same feature, and the layer has effectively fewer units. Random initialisation *breaks the symmetry*.

Derivation: the scale of the random weights. How large should the random weights be? Draw the W_{ij} of one layer independently with mean 0 and variance σ_w^2, independent of the incoming activations, which we take to have mean square $\mathbb{E}[h_j^2] = q$. Then, for a unit with H_{k-1} inputs (its *fan-in*) and zero bias,

$$\operatorname{Var}\left(a_i\right) = \sum_{j=1}^{H_{k-1}} \operatorname{Var}(W_{ij} h_j) = \sum_j \mathbb{E}[W_{ij}^2]\, \mathbb{E}[h_j^2] = H_{k-1}\, \sigma_w^2\, q,$$

since the products are uncorrelated with mean zero. The scale is preserved from layer to layer only if $H_{k-1}\sigma_w^2 \approx 1$ (for activations that behave like the identity near zero, such as tanh) — otherwise it is multiplied by $H_{k-1}\sigma_w^2$ in every layer and grows or decays geometrically with depth. The same argument for the backward pass (5), where W^{T} sums over the H_k units of the layer above (its *fan-out*), asks for $H_k \sigma_w^2 \approx 1$.

- *Glorot (Xavier) initialisation* [8] compromises between the two conditions with $\sigma_w^2 = 2/(H_{k-1} + H_k)$; drawn uniformly from $[-b, b]$, whose variance is $b^2/3$, this gives $b = \sqrt{6/(H_{k-1} + H_k)}$.
- *He initialisation* [10] accounts for ReLU, which zeroes half of a symmetric input: if a is symmetric about zero, $\mathbb{E}[\max(0, a)^2] = \frac{1}{2} \operatorname{Var}(a)$, so q is only half the previous layer's variance and the condition becomes $H_{k-1}\sigma_w^2 = 2$, i.e. $\sigma_w = \sqrt{2/H_{k-1}}$.

Algorithm 6 Random initialisation of a fully connected network

Require: layer widths $H_0, \dots, H_L$; scheme (Glorot for tanh/sigmoid, He for ReLU-type activations)

function INITIALISE($H_0, \dots, H_L$)
  for $k = 1, \dots, L$ **do**
    if Glorot **then** draw $W_{ij}^{(k)} \sim \mathcal{U}\big[-b, b\big]$, $b = \sqrt{6/(H_{k-1} + H_k)}$
    else draw $W_{ij}^{(k)} \sim \mathcal{N}\big(0, 2/H_{k-1}\big)$ ▷ He
    end if
    $\mathbf{b}^{(k)} \leftarrow \mathbf{0}$
  end for
  return θ
end function

Running time — initialisation

One random number per parameter: $O(p)$ time and the $O(p)$ memory of the parameters themselves. Negligible, but it decides whether the first thousand steps make progress.

Worked example — 7. Numbers for a first layer

A layer from $H_0 = 784$ inputs (a 28×28 image, standardised pixels) to $H_1 = 256$ units. *Naive*: $\sigma_w = 1$ gives $\mathrm{Var}(a) = 784$, a standard deviation of 28; nearly every tanh unit is saturated, with $g'(a) \approx 0$, and learns nothing. *Glorot*: $b = \sqrt{6/1040} = 0.0760$, variance $2/1040 = 0.00192$, so $\mathrm{Var}(a) = 784 \times 0.00192 = 1.51$. *He*: $\sigma_w = \sqrt{2/784} = 0.0505$, so $\mathrm{Var}(a) = 2$ and, after the ReLU, $\mathbb{E}[h^2] = 1$, the same mean square as the input.

Figure 3 follows the scale through 30 layers of width 256. For tanh, weights of standard deviation 1 keep the pre-activations at a root mean square of about 16 (85% of units saturated, $|a| > 3$), while 0.01 makes the signal vanish by a factor 0.16 per layer; Glorot keeps it of order one, decaying slowly (from 1.0 to 0.12 over 30 layers). For ReLU, Glorot's variance is half of what is needed and the mean square halves every layer, while He initialisation keeps it roughly constant (from 1.41 at layer 1 to 0.70 at layer 30; the residual drift comes from the finite width).

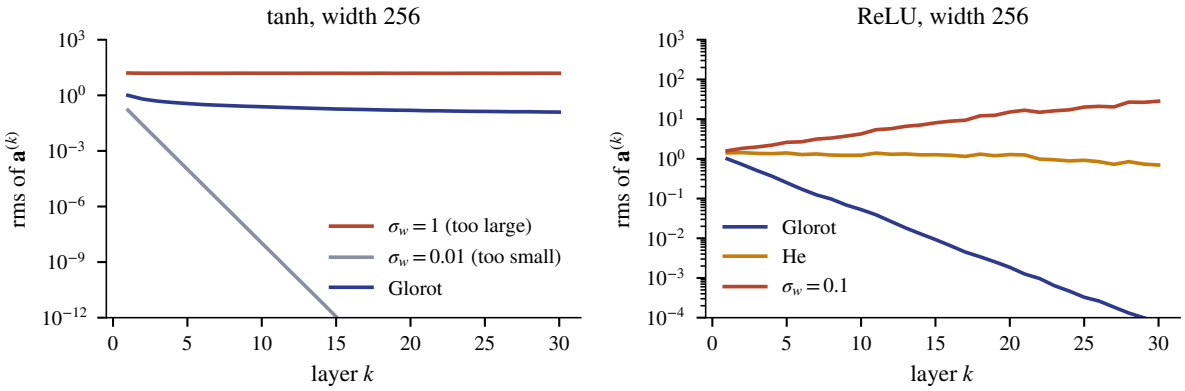


Figure 3. Root mean square of the pre-activations $\mathbf{a}^{(k)}$ through a 30-layer fully connected network of width 256 at initialisation (500 standard-normal inputs, zero biases, log scale). Left: tanh with weights of standard deviation 1, 0.01 and Glorot uniform. Right: ReLU with Glorot uniform, He, and a fixed standard deviation 0.1, slightly above He's $\sqrt{2/256} = 0.088$, which multiplies the expected mean square by $256 \times 0.01/2 = 1.28$ per layer.

In physics — signal propagation and the edge of chaos

The variance recursion above is a mean-field theory: for a wide network, the pre-activations of a layer are approximately Gaussian, and their variance q_k obeys a deterministic map $q_k = \sigma_w^2\, \mathbb{E}[g(\sqrt{q_{k-1}}\, z)^2] + \sigma_b^2$ with $z \sim \mathcal{N}(0,1)$, when the weights have variance σ_w^2/H_{k-1} and the biases variance σ_b^2. Poole *et al.* [18] and Schoenholz *et al.* [23] studied in the same way how the *correlation* between the signals of two different inputs evolves with depth. They found two phases, as in a magnet: an ordered phase, in which all inputs are mapped to nearly the same representation and gradients vanish, and a chaotic phase, in which nearby inputs are driven apart and gradients explode. Between them lies a critical line — the "edge of chaos" — where the depth scale over which information survives diverges, like a correlation length at a critical point. Good initialisations place the network at, or close to, criticality.

8 Choosing hyperparameters

A network comes with many hyperparameters: the width and depth (Note 11, Section 7.2), the activation, the learning rate and its schedule, the batch size, momentum or Adam's β's, the weight decay λ, the dropout rate, the number of epochs. They are chosen exactly as in Note 4, Section 7: train on D_{train}, compare on D_{valid}, and report the final error once on D_{test}. Two remarks:

- *Search as in Note 4, Section 7.2*: log-uniform random draws of η and λ (they act multiplicatively), many short runs over wide ranges first, then longer runs in the good region, stopping clearly bad runs early. With five or more hyperparameters a grid is hopeless [4].
- *The learning rate usually matters most.* In the random search of Figure 2 (right), the 20 trials with $\eta < 0.01$ all end between 14.3% and 17.85% validation error after 60 epochs, while the 13 trials with $\eta \geq 0.1$ have a median of 7.45%; the weight decay matters much less over the range searched. Hyperparameters interact: the best learning rate depends on the batch size, the optimiser, the initialisation and the activation.

9 Why training is hard

9.1 Non-convexity and symmetries

Note 11, Section 7.3, showed that the objective is not convex: permuting the hidden units of a layer (and, for tanh units, negating a unit's incoming weights, bias and outgoing weights) leaves the function unchanged, so every minimiser has $H!\,2^H$ equivalent copies, 3.7×10^9 for one tanh layer of 10 units. These copies are harmless — all are equally good — and only show that "the" minimum does not exist.

What does hurt are regions where the gradient is small but the loss is not: *plateaus*, where the loss barely changes over long stretches (for example while many units are saturated), and *saddle points*, which in high dimension are far more common than poor local minima [7] (Note 7, Section 4). On them, plain gradient descent crawls; momentum, adaptive step sizes and mini-batch noise help to move on. Empirically, for large networks the minima reached from different random initialisations are typically very similar in training loss, and usually in test loss, although they are different points in parameter space [9, Section 8.2].

9.2 Vanishing and exploding gradients

Unroll the recursion (5) from the top to layer 1. Each step multiplies by $W^{(k)\mathsf{T}}$ and by $D^{(k-1)} = \operatorname{diag}\big(g'(\mathbf{a}^{(k-1)})\big)$:

$$\boldsymbol{\delta}^{(1)} = D^{(1)} W^{(2)\mathsf{T}} D^{(2)} W^{(3)\mathsf{T}} \cdots D^{(L-1)} W^{(L)\mathsf{T}}\, \boldsymbol{\delta}^{(L)}, \qquad \big\|\boldsymbol{\delta}^{(1)}\big\| \leq \prod_{k=2}^{L} \Big(\big\|W^{(k)}\big\| \max_j \big|g'(a_j^{(k-1)})\big| \Big)\, \big\|\boldsymbol{\delta}^{(L)}\big\|, \tag{10}$$

with $\|W\|$ the largest singular value. The gradient reaching the first layer is a product of $L-1$ factors, and a product of many factors is typically exponentially small or exponentially large: *vanishing* or *exploding gradients* [3]. Sigmoid units are the worst case: $g' \leq \frac{1}{4}$, so unless the weights are large every layer divides the signal by at least four. For a chain of ten sigmoid units with weights 1, the factor is at most $(1/4)^{10} = 9.5 \times 10^{-7}$: the first layers receive essentially no gradient and do not learn. Exploding gradients appear when the factors exceed one, most dramatically in recurrent networks, where the same matrix is multiplied in at every time step; gradient clipping (Note 7, Section 7) limits

the damage.

Figure 4 measures the effect at initialisation in 30-layer networks of width 128. With sigmoid units and Glorot weights the gradient at layer 1 is 5.5×10^{-19} times that at layer 30; tanh with Glorot loses a factor of five; ReLU with He initialisation keeps it of order one (1.7 times the top value), since $g' = 1$ on the active units, and the residual network too (2.0 times). The remedies are therefore: activations whose derivative is 1 over a large range (ReLU and its smooth relatives GELU and SiLU), initialisation at the right scale (Section 7), and the two architectural changes of Sections 10.2 and 10.3.

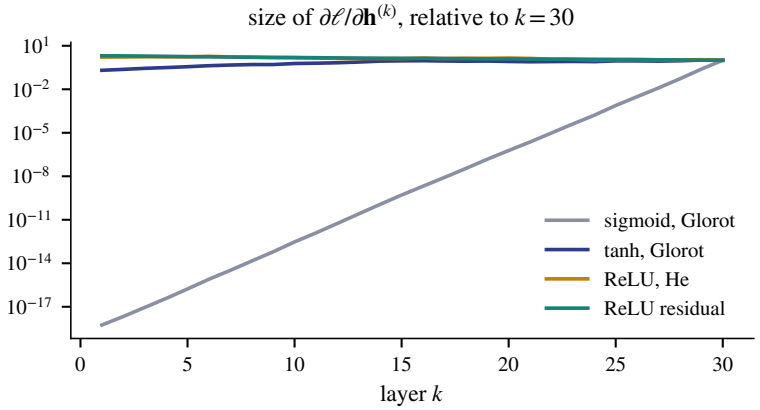


Figure 4. Root mean square of $\partial\ell/\partial\mathbf{h}^{(k)}$ in 30-layer networks of width 128 at initialisation (256 random inputs, a random linear loss on the last layer), divided by its value at layer 30. The residual network adds each layer's output to its input, $\mathbf{h}^{(k)} = \mathbf{h}^{(k-1)} + \mathrm{ReLU}(W^{(k)}\mathbf{h}^{(k-1)})$, with He weights divided by $\sqrt{30}$.

10 Practical tricks and the modern toolkit

10.1 Inputs, optimiser and learning rate

Standardise the inputs. Subtract the training mean of each input feature and divide by its standard deviation, with training statistics only (Note 1, Section 5.1, on leakage). Features of very different scales make the first-layer Hessian ill-conditioned, which slows gradient descent (Note 7, Section 3.2), and they break the assumptions of the initialisation formulas [13]. Regression targets are standardised for the same reason.

Optimiser. Mini-batches of 32 to a few thousand examples exploit vectorised hardware (Note 7, Section 5). SGD with momentum (Note 7, Section 6.1) remains strong for convolutional networks; Adam and AdamW (Note 7, Section 6.4) are the default elsewhere and are less sensitive to the learning rate. Newton's method (Note 7, Section 6.2) is out of reach: the Hessian of a network with $p = 10^7$ parameters would have 10^{14} entries, and it is not positive definite away from a minimum.

Learning-rate schedule. The learning rate is decreased during training with one of the schedules of Note 7, Section 3.3; the Robbins–Monro conditions of Note 7, Section 5.3, say why a decay is needed for convergence with noisy gradients. For example $\eta_t = \eta_0/t^{\delta}$ satisfies both conditions exactly when $\frac{1}{2} < \delta \le 1$: the sum of $t^{-\delta}$ diverges for $\delta \le 1$, and the sum of $t^{-2\delta}$ converges for $\delta > \frac{1}{2}$. Warm-up matters most for deep networks with adaptive optimisers, because Adam's second-moment estimate rests on very few gradients in the first steps. A rate that is too large can do more than oscillate: in the left panel of Figure 2, one huge step with $\eta = 3$ kills every ReLU unit of the second hidden layer, and the network is left predicting a constant (loss ≈ 0.70; compare Question A5).

10.2 Normalisation layers

Initialisation fixes the scale of the signals at the start; normalisation layers keep it fixed throughout training. *Batch normalisation* [12] standardises each unit's pre-activation over the mini-batch and then restores a learned scale and shift:

$$\hat{a}_{ij} = \frac{a_{ij} - \mu_j}{\sqrt{s_j^2 + \epsilon}}, \qquad \mu_j = \frac{1}{n'} \sum_{i \in B} a_{ij}, \quad s_j^2 = \frac{1}{n'} \sum_{i \in B} (a_{ij} - \mu_j)^2, \qquad \tilde{a}_{ij} = \gamma_j \, \hat{a}_{ij} + \beta_j, \tag{11}$$

for example i and unit j, with a small ϵ (e.g. 10^{-5}) against division by zero. The parameters γ_j, β_j are trained with the rest, so the layer can represent any scale, including the identity; the bias of the preceding affine layer becomes redundant (the mean is subtracted) and is dropped. At test time a prediction must not depend on the other examples in the batch, so μ_j and s_j^2 are replaced by running averages accumulated during training. Batch normalisation allows much larger learning rates; it was introduced to reduce shifts of the input distribution of each layer during training, and later work argues that its main benefit is a smoother loss landscape [22]. Its dependence on the batch is also its weakness: small batches give noisy statistics, and training and test behaviour differ.

Layer normalisation [1] applies the same standardisation to the H units of *one* example, with μ_i and s_i^2 computed over j; it behaves identically for any batch size, at training and test time, and is the normalisation of transformers. RMSNorm divides by the root mean square without subtracting the mean, a cheaper variant [27] used in several recent language models [25].

Algorithm 7 Batch normalisation and layer normalisation (forward pass)

Require: pre-activations $A \in \mathbb{R}^{n' \times H}$; scale and shift $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^H$; $\epsilon > 0$; running averages $\bar{\mu}, \bar{s}^2$ and their decay ρ (batch norm only)

function BatchNorm(A, *training*)
  if *training* **then**
    $\mu_j \leftarrow$ mean of column j, $s_j^2 \leftarrow$ variance of column j ▷ over the n' examples
    $\bar{\mu} \leftarrow \rho\bar{\mu} + (1-\rho)\mu$, $\bar{s}^2 \leftarrow \rho\bar{s}^2 + (1-\rho)s^2$ ▷ for inference
  else $\mu \leftarrow \bar{\mu}$, $s^2 \leftarrow \bar{s}^2$
  end if
  return $\tilde{A}_{ij} = \gamma_j (A_{ij} - \mu_j)/\sqrt{s_j^2 + \epsilon} + \beta_j$
end function
function LayerNorm(A) ▷ same at training and test time
  $\mu_i \leftarrow$ mean of row i, $s_i^2 \leftarrow$ variance of row i ▷ over the H units
  return $\tilde{A}_{ij} = \gamma_j (A_{ij} - \mu_i)/\sqrt{s_i^2 + \epsilon} + \beta_j$
end function

Running time — normalisation layers

Forward: $O(n'H)$ per layer (two passes for mean and variance, one to normalise); backward, obtained by automatic differentiation, also $O(n'H)$. Both are small next to the $O(n'HH')$ matrix product of the layer. Memory: the normalised values and the n' or H statistics for the backward pass, $2H$ parameters, and for batch norm $2H$ running averages. Prediction: $O(H)$ per example (batch norm at inference is an affine map and can be folded into the preceding weights).

Worked example — 8. Normalising four numbers

One unit has the pre-activations $(1, 2, 3, 6)$ on a mini-batch of four examples. Mean $\mu = 3$, variance $s^2 = \frac{1}{4}(4 + 1 + 0 + 9) = 3.5$, standard deviation 1.871 (take $\epsilon \approx 0$). The normalised

values are $(-1.069, -0.535, 0, 1.604)$: mean 0, variance 1. With $\gamma = 2$ and $\beta = 1$ the layer outputs $(-1.138, -0.069, 1, 4.207)$. Layer normalisation does the same arithmetic on the four units of *one* example: if an example's pre-activations were $(1, 2, 3, 6)$, it would produce the same numbers if every unit had $\gamma_j = 2$ and $\beta_j = 1$, whatever the other examples in the batch.

10.3 Residual connections

A *residual block* [11] adds its input to its output:

$$\mathbf{h}^{(k)} = \mathbf{h}^{(k-1)} + F_k\big(\mathbf{h}^{(k-1)}\big), \tag{12}$$

where F_k is a small sub-network (for example normalisation, affine, activation, affine). The block learns the *change* F_k to the representation rather than a whole new representation, and if $F_k = 0$ it is the identity, so adding blocks can never make the best achievable fit worse.

Derivation: an identity path for the gradient. Differentiating (12), $\partial\mathbf{h}^{(k)}/\partial\mathbf{h}^{(k-1)} = I + \partial F_k/\partial\mathbf{h}^{(k-1)}$, so the backward recursion reads

$$\frac{\partial \ell}{\partial \mathbf{h}^{(k-1)}} = \frac{\partial \ell}{\partial \mathbf{h}^{(k)}} + \Big(\frac{\partial F_k}{\partial \mathbf{h}^{(k-1)}}\Big)^{\mathsf{T}} \frac{\partial \ell}{\partial \mathbf{h}^{(k)}}.$$

The first term copies the gradient unchanged to the layer below. Unrolling over all blocks, the product of Jacobians $\prod_k (I + J_k)$ expands into a sum of products of all subsets of the J_k, including the term I: the gradient at the bottom contains $\partial\ell/\partial\mathbf{h}^{(L)}$ itself, however deep the network. In one dimension with twenty blocks, each of slope $J_k = 0.1$, a plain chain multiplies the gradient by $0.1^{20} = 10^{-20}$, the residual chain by $1.1^{20} = 6.7$; with $J_k = -0.1$ the residual factor is $0.9^{20} = 0.12$ — smaller, but not exponentially small in the way the plain chain is. Residual connections made networks of a hundred layers and more trainable [11]; Note 13, Section 7, shows them in convolutional networks.

Algorithm 8 A pre-normalisation residual MLP block (forward and backward)

Require: block input $X \in \mathbb{R}^{n' \times H}$; LayerNorm parameters; $W_1 \in \mathbb{R}^{H_{\mathrm{f}} \times H}$, $W_2 \in \mathbb{R}^{H \times H_{\mathrm{f}}}$ and biases; activation g (e.g. GELU)

function ResBlockForward(X)
  $Z \leftarrow \textsc{LayerNorm}(X)$; $A \leftarrow ZW_1^{\mathsf{T}} + \mathbf{1}\mathbf{b}_1^{\mathsf{T}}$; $U \leftarrow g(A)$; $F \leftarrow UW_2^{\mathsf{T}} + \mathbf{1}\mathbf{b}_2^{\mathsf{T}}$
  return $X + F$ ▷ the skip connection
end function
function ResBlockBackward($\bar{Y}$) ▷ $\bar{Y} = \partial\ell/\partial(\text{output})$
  $\bar{F} \leftarrow \bar{Y}$; gradients of $W_2, \mathbf{b}_2$ from $\bar{F}$ and U as in (7); $\bar{A} \leftarrow (\bar{F}W_2) \odot g'(A)$
  gradients of $W_1, \mathbf{b}_1$ from $\bar{A}$ and Z; $\bar{Z} \leftarrow \bar{A}W_1$; $\bar{X}_F \leftarrow$ LayerNorm backward of $\bar{Z}$
  return $\bar{X} = \bar{Y} + \bar{X}_F$ ▷ identity path plus branch
end function

Running time — residual MLP block

Forward: two matrix products, $O(n'HH_{\mathrm{f}})$ each, plus $O(n'(H + H_{\mathrm{f}}))$ for the normalisation, activation and addition; backward about twice the forward. Parameters: $2HH_{\mathrm{f}} + H_{\mathrm{f}} + 3H$. The skip connection itself costs $O(n'H)$ and no parameters. In transformers $H_{\mathrm{f}} = 4H$ is the usual choice.

Modern connection — the training recipe of a transformer

A transformer layer consists of two residual blocks, $\mathbf{x} \leftarrow \mathbf{x} + \text{Attention}(\text{LN}(\mathbf{x}))$ and $\mathbf{x} \leftarrow \mathbf{x} + \text{MLP}(\text{LN}(\mathbf{x}))$, where the MLP is the GELU block of Algorithm 8. Putting the normalisation inside the branch ("pre-LN"), so that the skip path is a pure identity, trains more stably than the original arrangement with the normalisation after the addition and tolerates shorter warm-ups [26]. The rest of the recipe — AdamW, warm-up and cosine decay, gradient clipping — is that of Note 7, Section 7; what this note adds is the initialisation, the placement of the normalisation, mixed precision, and weight decay that skips biases and normalisation parameters. Nothing in this pipeline is specific to language: the same recipe trains vision transformers and the networks used for physics surrogates.

10.4 A debugging checklist

Networks fail silently. The following order of checks catches most problems before an expensive run.

1. *Look at the data and the pipeline.* Plot inputs and labels after preprocessing; check shapes, units, standardisation with training statistics, and that labels are aligned with inputs.
2. *Check the initial loss.* With small output weights an m-class softmax predicts nearly uniform probabilities, so the initial cross-entropy should be close to $\ln m$ (2.303 for ten classes). A much larger value means badly scaled outputs or a wrong loss.
3. *Check the gradient* on a tiny network (Section 4) if any backward code was written by hand.
4. *Overfit a tiny subset.* Train on 10 to 100 examples without regularisation. The training loss must go close to zero; if it does not, the model, the loss or the optimiser is broken, and no amount of data will fix it.
5. *Find the learning rate.* Try a few values a factor of three to ten apart; take the largest that decreases the loss smoothly (Figure 2, left).
6. *Watch the curves.* Plot training and validation loss per epoch (Figure 2, centre). Training loss high: underfitting — bigger model, longer training, higher learning rate. Gap growing: overfitting — more data or augmentation, weight decay, dropout, early stopping. Also monitor the gradient norm and the fraction of inactive ReLU units per layer.
7. *Only then scale up*, and change one thing at a time.

In physics — physics-informed training

Automatic differentiation computes derivatives not only with respect to parameters but also with respect to *inputs*. Physics-informed neural networks [20] exploit this: a network $u_\theta(x,t)$ represents the solution of a differential equation, say $\partial_t u = \nu\,\partial_x^2 u$, and the training loss adds to the data misfit the squared residual $\left(\partial_t u_\theta - \nu\,\partial_x^2 u_\theta\right)^2$ at many collocation points, with the derivatives obtained by differentiating the network through its graph. Training then uses exactly Algorithm 4, and all the issues of this note reappear — the residual and data terms have very different scales and gradient magnitudes, and balancing them is one of the main practical difficulties of the method. Smooth activations such as tanh or SiLU are used, because the second derivative of a ReLU network is zero almost everywhere.

Summary Training a network is regularised empirical risk minimisation with mini-batch gradients. For matched output/loss pairs the output error is $\mathbf{y} - \mathbf{t}$. Backpropagation applies the chain rule in reverse order: the error of layer $k-1$ is $g'(\mathbf{a}^{(k-1)}) \odot W^{(k)\mathsf{T}}\boldsymbol{\delta}^{(k)}$ and the weight gradient is the outer product $\boldsymbol{\delta}^{(k)}\mathbf{h}^{(k-1)\mathsf{T}}$; a training step costs about three forward passes, and the price is the memory of the stored activations. Automatic differentiation does the same for any graph of modules; central differences check it. Weight decay shrinks weights (not biases) by $1-\eta\lambda$ per step, L_1 drives some to zero, dropout injects scaled multiplicative noise, and early stopping limits the number of steps. Weights must be random, to break symmetry, with variance about $2/(\text{fan-in} + \text{fan-out})$ (Glorot) or $2/\text{fan-in}$ for ReLU (He). The loss is non-convex and full of symmetric copies, plateaus and saddles; gradients vanish or explode as products of many Jacobians. ReLU-type activations, good initialisation, normalisation layers and residual connections keep the gradients alive, and a disciplined checklist — initial loss, tiny-subset overfitting, learning-rate sweep, learning curves — keeps the training honest.

Questions and exercises

Questions are sorted by kind. Parts A and B are of the kind used in the midterm examination: they test understanding, interpretation and short calculations (at most simple derivatives), not long proofs. Parts C and D go further and are recommended for practice and for the project.

A. Conceptual questions

A1. List the four ingredients that stochastic gradient descent needs before it can train a neural network, and say in one sentence why each is needed.

A2. Every row of the softmax Jacobian sums to zero, and the components of $\mathbf{y} - \mathbf{t}$ sum to zero. Explain why these are two statements of the same fact.

A3. In Worked example 2 the input x has two children. What would go wrong if the backward sweep overwrote $\bar{x}$ instead of accumulating it, and why does the same accumulation make shared weights correct?

A4. What must be stored during the forward pass so that the backward pass can run? How does this memory grow with the batch size and the depth, and what does activation checkpointing trade for it?

A5. In Worked example 3 hidden unit 1 receives no gradient. Under which condition does this persist across all training examples, and can the unit then recover? How do He initialisation and a small positive bias change the odds, and how can a too-large learning rate cause the situation (Figure 2, left)?

A6. In gradient checking, why are central differences preferred to forward differences, why is a relative error used, and why should dropout be switched off? Why may a correct implementation disagree on a few coordinates with ReLU units?

A7. Why are biases usually excluded from weight decay? Why must weight decay be decoupled from the gradient when Adam is used?

A8. Compare what the L_2 and L_1 penalties do to a weight whose data gradient is zero under repeated gradient steps. Why does the L_1 penalty need a clip-at-zero update to produce exact zeros?

A9. In inverted dropout, why are the surviving activations divided by $1 - p_{\mathrm{d}}$? What would go wrong at test time if a network trained with dropout but without this rescaling were used without masks?

A10. Explain why a tanh network cannot be trained from all-zero weights, and why initialising all weights to the same non-zero constant is not much better.

A11. State the variance condition behind Glorot initialisation and explain where the extra factor 2 in He initialisation comes from.

A12. Why are plateaus and saddle points, rather than poor local minima, the main obstacle when training large networks, and which ingredients of stochastic gradient training help to leave them?

A13. Using (10), explain why deep networks of sigmoid units suffer from vanishing gradients, and list three remedies discussed in this note.

A14. How does batch normalisation behave differently at training and at test time, and why? Give two reasons why layer normalisation is preferred in transformers.

A15. Explain, using the Jacobian of a residual block, why gradients reach the first layers of a very deep residual network.

A16. A ten-class classifier starts training with a cross-entropy of 23 instead of about 2.3. What does this suggest? In what order would you then carry out the checks of Section 10.4, and why is overfitting a tiny subset a useful test?

A17. In the random search of Figure 2 (right), which hyperparameter mattered most? Why do interactions between hyperparameters (for example the learning rate and the batch size) argue against tuning them one at a time?

A18. Explain why the number of training epochs acts as a capacity control. In Algorithm 4, what does the patience P control, and why is θ_{best} returned rather than the final θ? In the centre panel of Figure 2, why would monitoring the validation loss with a small patience stop too early?

A19. Why does mixed-precision training keep a 32-bit copy of the weights, and why does multiplying the loss by a constant before the backward pass not change the direction of the gradient?

B. Short calculations

B1. A softmax output is $\mathbf{y} = (0.1, 0.2, 0.7)$. Write the 3×3 Jacobian $\partial \mathbf{y}/\partial \mathbf{a}$ and verify that its rows sum to zero. For the true class 3, compute $\partial \ell/\partial \mathbf{y}$ for the cross-entropy, multiply it by the Jacobian to obtain $\boldsymbol{\delta} = \partial \ell/\partial \mathbf{a}$, and check the result against $\mathbf{y} - \mathbf{t}$.

B2. A 2–2–1 network has ReLU hidden units with $W^{(1)} = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$, $\mathbf{b}^{(1)} = \mathbf{0}$, and a sigmoid output with $W^{(2)} = (2, 1)$, $b^{(2)} = -1$, trained with the binary cross-entropy. For $\mathbf{x} = (2, 1)$ and $t = 0$ carry out the forward pass (pre-activations, activations, output, loss) and the backward pass: $\delta^{(2)}$, $\boldsymbol{\delta}^{(1)}$, and the gradients of all weights and biases. Which parameters receive a zero gradient, and why?

B3. For $f(x, y, z) = (x + y)\, z + x^2$ draw the computational graph with intermediate nodes $u = x + y$, $v = uz$, $w = x^2$, $f = v + w$. At $(x, y, z) = (1, 2, -1)$ compute all values in a forward sweep and

all adjoints in one backward sweep, and check them against the partial derivatives computed directly.

B4. A backprop gradient $\mathbf{g}_{\text{bp}} = (0.50, -1.20, 0.003)$ is compared with a finite-difference gradient $\mathbf{g}_{\text{fd}} = (0.50, -1.19, 0.004)$. Compute r of (8) over the three coordinates, and the same ratio $|g_{\text{bp}} - g_{\text{fd}}|/(|g_{\text{bp}}| + |g_{\text{fd}}|)$ for each coordinate separately. Which coordinate is suspicious, and how could a ReLU kink explain a disagreement on one coordinate even when the code is correct?

B5. Return to Worked example 3 and take a step with $\eta = 0.1$ instead of 0.05. Compute the new parameters, the new forward pass and the new loss. What happened to hidden unit 2, and what gradient will its weights receive at the next step on this example?

B6. A weight $w = 0.3$ has zero data gradient. With $\eta = 0.1$ and $\lambda = 1$, how many steps does it take for the L_1 penalty to set it to zero, and what is its value after the same number of steps with weight decay (9)?

B7. A fully connected layer has 512 inputs and 512 outputs. Compute the Glorot bound b and the He standard deviation. If instead the weights have standard deviation 0.1 and the inputs have unit mean square, what is the standard deviation of a pre-activation? With this standard deviation, by what factor does the mean square $\mathbb{E}[h^2]$ of the hidden activations grow per ReLU layer (use $\mathbb{E}[h^2] = \frac{1}{2}\operatorname{Var}(a)$), and after how many layers has it grown by more than 10^3?

B8. One unit of a batch-normalisation layer receives the pre-activations $(2, 4, 4, 6)$ on a mini-batch, and has $\gamma = 3$, $\beta = -1$, $\epsilon \approx 0$. Compute the batch mean and variance, the normalised values and the outputs. At test time the running averages are $\bar{\mu} = 3.8$ and $\bar{s}^2 = 2.25$; what is the output for a pre-activation of 5?

C. Derivations

C1. Derive the Jacobian $\partial \mathbf{y}/\partial \mathbf{a} = \operatorname{diag}(\mathbf{y}) - \mathbf{y}\mathbf{y}^{\mathsf{T}}$ of the softmax and use it to show $\partial \ell/\partial \mathbf{a} = \mathbf{y} - \mathbf{t}$ for the cross-entropy. Show that for $m = 2$ the result reduces to the sigmoid case $y - t$ of Note 9.

C2. Starting from the per-example recursion (5)–(6), derive the mini-batch equations (7) for the mean loss, and count the multiply-adds of the forward and backward passes to show that a training step costs about three forward passes.

C3. Derive the variance recursion for the pre-activations of a network with independent zero-mean weights. Show that $\mathbb{E}[\max(0, a)^2] = \frac{1}{2}\operatorname{Var}(a)$ when a is symmetric about zero, and obtain the He condition. Repeat the argument for the backward pass to obtain the fan-out condition, and explain Glorot's compromise.

C4. Prove that gradient descent started from all-zero weights and biases in a tanh network changes only the output bias, and that with a small enough learning rate it converges to a stationary point of the training objective. Then prove by induction that two hidden units with identical incoming and outgoing weights remain identical under gradient descent.

C5. For a stack of L residual blocks (12) with Jacobians $J_k = \partial F_k/\partial \mathbf{h}^{(k-1)}$, write $\partial \ell/\partial \mathbf{h}^{(0)}$ as a product and expand it as a sum over subsets of blocks. Compare with a plain stack, and discuss what happens when all J_k are small.

D. Programming

D1. Implement Algorithm 1 in numpy for an MLP with ReLU hidden layers and a softmax output, and verify it with Algorithm 3. Then introduce three deliberate bugs (a missing g', a missing transpose that happens to have the right shape for square layers, a batch mean replaced by a sum) and report the relative error each produces.

D2. Reproduce Figures 3 and 4 in numpy. Add batch normalisation (Algorithm 7) before each activation of the sigmoid network and measure again how the gradient changes with depth.

D3. In PyTorch, train an MLP on the two-moons data (or MNIST) with AdamW, linear warm-up and cosine decay. (a) First check the initial loss and overfit 50 examples. (b) Run a random search of 30 trials over $\log_{10}\eta$, $\log_{10}\lambda$ and the dropout rate, and plot the validation error against each hyperparameter. Which matters most? (c) Compare the validation curves of the best setting with and without weight decay and dropout.

Further reading. Goodfellow, Bengio and Courville [9], chapters 6–8; Prince [19], chapters 7–9 and 11; Bishop [5], chapter 5; Murphy [16], chapter 13; Baydin *et al.* [2] on automatic differentiation; LeCun *et al.* [13] for practical advice that is still valid.

References

[1] J. L. Ba, J. R. Kiros and G. E. Hinton, Layer normalization, arXiv:1607.06450 (2016).

[2] A. G. Baydin, B. A. Pearlmutter, A. A. Radul and J. M. Siskind, Automatic differentiation in machine learning: a survey, *J. Mach. Learn. Res.* 18(153), 1–43 (2018).

[3] Y. Bengio, P. Simard and P. Frasconi, Learning long-term dependencies with gradient descent is difficult, *IEEE Trans. Neural Netw.* 5(2), 157–166 (1994).

[4] J. Bergstra and Y. Bengio, Random search for hyper-parameter optimization, *J. Mach. Learn. Res.* 13, 281–305 (2012).

[5] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[6] T. Chen, B. Xu, C. Zhang and C. Guestrin, Training deep nets with sublinear memory cost, arXiv:1604.06174 (2016).

[7] Y. N. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli and Y. Bengio, Identifying and attacking the saddle point problem in high-dimensional non-convex optimization, *NeurIPS* (2014).

[8] X. Glorot and Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, *Proc. AISTATS*, 249–256 (2010).

[9] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press (2016).

[10] K. He, X. Zhang, S. Ren and J. Sun, Delving deep into rectifiers: surpassing human-level performance on ImageNet classification, *Proc. ICCV*, 1026–1034 (2015).

[11] K. He, X. Zhang, S. Ren and J. Sun, Deep residual learning for image recognition, *Proc. CVPR*, 770–778 (2016).

[12] S. Ioffe and C. Szegedy, Batch normalization: accelerating deep network training by reducing internal covariate shift, *ICML* (2015).

[13] Y. LeCun, L. Bottou, G. B. Orr and K.-R. Müller, Efficient BackProp, in *Neural Networks: Tricks of the Trade*, Lecture Notes in Computer Science 1524, Springer, 9–50 (1998).

[14] I. Loshchilov and F. Hutter, Decoupled weight decay regularization, *ICLR* (2019).

[15] P. Micikevicius *et al.*, Mixed precision training, *ICLR* (2018).

[16] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press (2022).

[17] R.-E. Plessix, A review of the adjoint-state method for computing the gradient of a functional with geophysical applications, *Geophys. J. Int.* 167, 495–503 (2006).

[18] B. Poole, S. Lahiri, M. Raghu, J. Sohl-Dickstein and S. Ganguli, Exponential expressivity in deep neural networks through transient chaos, *NeurIPS* (2016).

[19] S. J. D. Prince, *Understanding Deep Learning*, MIT Press (2023).

[20] M. Raissi, P. Perdikaris and G. E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378, 686–707 (2019).

[21] D. E. Rumelhart, G. E. Hinton and R. J. Williams, Learning representations by back-propagating errors, *Nature* 323, 533–536 (1986).

[22] S. Santurkar, D. Tsipras, A. Ilyas and A. Mądry, How does batch normalization help optimization?, *NeurIPS* (2018).

[23] S. S. Schoenholz, J. Gilmer, S. Ganguli and J. Sohl-Dickstein, Deep information propagation, *ICLR* (2017).

[24] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, Dropout: a simple way to prevent neural networks from overfitting, *J. Mach. Learn. Res.* 15, 1929–1958 (2014).

[25] H. Touvron *et al.*, LLaMA: open and efficient foundation language models, arXiv:2302.13971 (2023).

[26] R. Xiong *et al.*, On layer normalization in the transformer architecture, *ICML* (2020).

[27] B. Zhang and R. Sennrich, Root mean square layer normalization, *NeurIPS* (2019).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.

The IFT 6390 slides on training neural networks are based on material by Hugo Larochelle.